

Computational Intelligence

Projekt-Präsentationen

Domenico Avolio, Tim Lukas, Maximilian Murrath

Hochschule für Technik Stuttgart

19. Juni 2026

Genetischer Algorithmus: Grundidee des Inselmodells

Konzept der unabhängigen Inseln

- ▶ Mehrere unabhängige Populationen statt einer einzigen.
- ▶ Jede Insel besitzt eine **eigene Kultur** (Konfiguration, Selektion, Crossover, Mutation).
- ▶ Parallele Suchprozesse auf mehreren CPU-Kernen (Multithreading).

Insel-Zyklus (wiederholt): 1. Selektion → 2. Crossover → 3. Mutation → 4. Lokale Suche → 5. Elitismus.

Warum ein Inselmodell?

- ▶ **Vorteile:** Reduziert Gefahr früher Konvergenz, erhält mehrere Suchrichtungen, erhöht genetische Diversität.

Wichtige Implementierungsentscheidung

- ▶ Jede Insel verwendet unterschiedliche Operatoren (z. B. Tournament vs. Roulette, PMX vs. Order Crossover).
- ▶ **Begründung:** Es existiert kein universell bester genetischer Operator. Durch die parallele Verwendung können unterschiedliche Bereiche des Suchraums effizienter untersucht werden.

Genetischer Algorithmus: Generationswechsel

Erzeugung einer neuen Generation

- ▶ **Ablauf:** 1. Elitismus (Bestes Individuum), 2. Elternwahl, 3. Crossover, 4. Mutation (p_m), 5. Lokale Suche (p_{ls}), 6. Fitness berechnen.

Warum Elitismus?

- ▶ Bereits gefundene gute Lösungen gehen niemals verloren. Stabilisiert den Suchprozess.

Adaptive Mutationsrate

- ▶ **Auslöser:** Sinkt die Diversität oder stagniert die Fitness, wird die Mutationsrate automatisch erhöht.
- ▶ **Ziel:** Dynamischer Kompromiss zwischen Exploration und Exploitation.

Lokale Suche (Memetic Algorithm)

- ▶ **Operationen:** Positionen tauschen, Segmente mischen, Neustarts, Akzeptanz schlechterer Lösungen (Simulated Annealing).
- ▶ **Begründung:** Genetische Operatoren für globale Suche, gepaart mit lokaler Suche zur gezielten Verfeinerung vielversprechender Lösungen.

Genetischer Algorithmus: Migration & Olympia

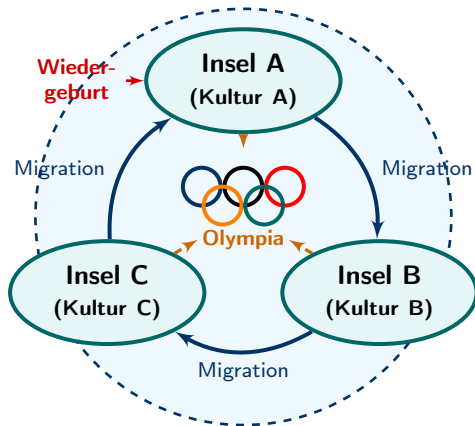
Migration & Olympische Spiele

- ▶ **Ring-Migration:** Weitergabe der besten Individuen im festen Intervall. Gute Lösungen verbreiten sich im Archipel.
- ▶ **Olympische Spiele:** Globale Wettkämpfe der Insel-Champions. Gewinner wird auf alle Inseln kopiert.

Selbstanpassung

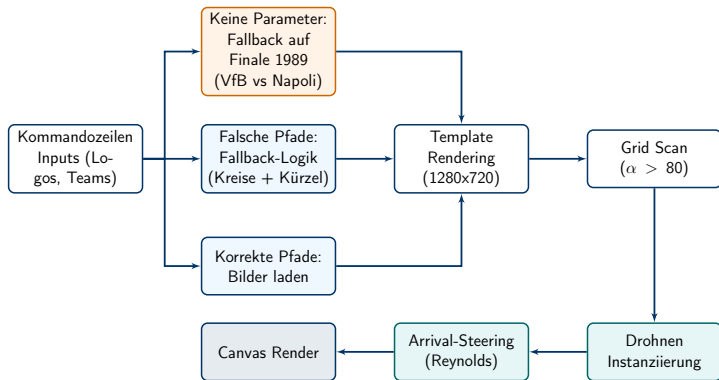
- ▶ **Inselsterben & Wiedergeburt:** Bei langer Stagnation stirbt eine Insel und wird mit neuen, zufälligen Parametern neu erzeugt.
- ▶ **Mehrstufige Adaptivität:** Individuum (Mutation, Lokale Suche), Population (Migration, Diversität), Insel (Wiedergeburt, Parameter).

Bestes gefundenes Ergebnis (Makespan): 157724



Drohnen Fußball-Show: Grundidee & Ablauf

Grundidee: Reale Fußball-Ergebnisse beliebiger Vereine als beeindruckende Drohnen-Formation am Nachthimmel simulieren.



Drohnen Fußball-Show: Zuweisung von Farbe & Koordinaten

Das „Malen nach Zahlen“-Prinzip: Das Bild wird auf ein unsichtbares Raster gelegt. Jede Farb-Zelle entspricht genau **einer** Drohne.



- **Schritt 1 (Raster):** Bild wird in ein festes Gitter (Grid) unterteilt.
- **Schritt 2 (Prüfung):** Ist die Zelle transparent? → Ignorieren. Enthält sie Farbe? → Drohne erzeugen!

Drohne 1
Ziel-X: 100
Ziel-Y: 100
Farbe: **Blau**

Drohne 2
Ziel-X: 200
Ziel-Y: 200
Farbe: **Weiß**

Drohnen Fußball-Show: Physik & Kollisionsvermeidung

Startbedingungen: Drohnen spawnen zufällig am **unteren Bildschirmrand** und erhalten dort ihr Endziel.

1. Separation (Abstoßung)

- ▶ Drohnen stoßen sich ab, wenn sie zu nahe sind.
- ▶ **Formel:** $\vec{F}_{sep} = \frac{\vec{p\ddot{o}s}_{this} - \vec{p\ddot{o}s}_{other}}{d^2}$
- ▶ **Bedeutung:** Je näher, desto extremer die Abstoßung!

Spatial Partitioning (Optimierung)

- ▶ Raum wird in ein Raster (**SpatialGrid**) unterteilt.
- ▶ Nur direkte Nachbarn werden geprüft → extrem schnell! ($O(N)$)

Legende: $\vec{F}_{sep}, \vec{F}_{evade}$ = Berechnete Flug-Kräfte | $\vec{p\ddot{o}s}_{this}$ = Eigene Pos. | $\vec{p\ddot{o}s}_{other}$ = Andere Drohne
 $\vec{p\ddot{o}s}_{broadcaster}$ = Blockierte Drohne | d = Distanz | C = Ausweich-Stärke

2. Blockaden & Ausweichen

- ▶ **Problem:** Fliegende Drohne bleibt stecken.
- ▶ **Lösung:** Pulsiert rot, sendet Notsignal.
- ▶ **Formel:** $\vec{F}_{evade} = \left(\frac{\vec{p\ddot{o}s}_{this} - \vec{p\ddot{o}s}_{broadcaster}}{d} \right) \times C$
- ▶ **Bedeutung:** Geparkte weichen weg von der blockierten Drohne aus und bilden eine rettende Gasse.

F1 AI Racing: Grundidee

- ▶ **Die Simulation:** Ein vollautomatisches Formel-1-Rennen, in dem virtuelle Agenten (Autos) das Fahren komplett von Null auf erlernen.
- ▶ **Der Startpunkt (Anfänger):** In der ersten Generation haben die Autos keinerlei Vorwissen über Lenkung, Bremsen oder die Strecke. Sie fahren erratisch und scheiden sofort aus.
- ▶ **Die Entwicklung (Evolution):** Über Tausende von Generationen entwickeln sich die Agenten durch Try and Error weiter, bis sie wie Profis Ideallinien fahren und Windschatten strategisch nutzen.

F1 AI Racing: Architekturwahl

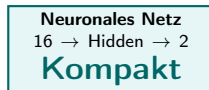
Warum kein Q-Learning (Q-Tabelle)?

- ▶ Q-Learning erfordert diskrete Zustände (Raster).
- ▶ **Das F1-Problem:**
 - ▶ Exakte X/Y Koordinaten
 - ▶ Stufenlose Sensordistanzen
 - ▶ Fließkomma-Geschwindigkeiten
- ▶ $\Rightarrow$ **State-Space Explosion!** Eine Q-Tabelle für alle diese Zustände wäre unendlich groß und würde sofort abstürzen.



Lösung: Neuroevolution

- ▶ Statt jeden Zustand starr zu speichern, nutzen wir ein **Neurales Netz** als universellen Funktionsapproximator.
- ▶ Es lernt durch **Genetische Algorithmen** (Evolution statt Q-Values).



F1 AI Racing: Wahrnehmung der KI

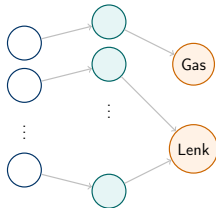
1. Laser Sensoren (Raycasting)

- ▶ Das Auto schießt 7 simulierte Laserstrahlen in verschiedenen Winkeln (W) nach vorne und zur Seite ab, um die Fahrbahnbegrenzung zu erkennen.
- ▶ **Formel (Ray-Wachstum in 3-Pixel-Schritten):**
 $\text{Step}_X = \cos(W) \cdot 3$ und $\text{Step}_Y = \sin(W) \cdot 3$

2. Checkpoint Radar

- ▶ Das Auto orientiert sich mathematisch am nächsten Checkpoint.
- ▶ **Distanz (Satz des Pythagoras):**
$$\text{Distanz} = \sqrt{(\text{Target}_X - \text{Car}_X)^2 + (\text{Target}_Y - \text{Car}_Y)^2}$$
- ▶ **Winkel (Trigonometrie):**
$$\text{Winkel} = \arctan2(\text{Target}_Y - \text{Car}_Y, \text{Target}_X - \text{Car}_X) - \text{Heading}$$

F1 AI Racing: Das Gehirn (Neuronales Netz)



Legende:

S_i = Sensor-Input H_i = Hidden Neuron

Aktivierungsfunktionen:

- **Gas (Sigmoid):** $f(x) = \frac{1}{1+e^{-x}}$
Mappt auf $[0 \dots 1]$ (Kein Rückwärts!)
- **Lenkung (Tanh):** $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
Mappt auf $[-1 \dots 1]$ (Links/Rechts)

Im Hintergrund: Gewichtsmatrizen

Jede Kante ist eine reelle Zahl (Gewicht). Bei der Fortpflanzung werden diese Zahlen mutiert (Ersatz für die Q-Tabelle!).

Matrix 1: Input → Hidden

Sensoren	H_1	H_2	...	H_{16}
S_1 (Laser)	0.45	-1.2	...	0.02
S_2 (Laser)	0.88	0.11	...	-0.5
⋮	⋮	⋮	⋮	⋮
S_{16} (Radar)	-0.9	0.44	...	1.33

Matrix 2: Hidden → Output

Neuronen	Gas	Lenkung
H_1	0.77	-0.1
H_2	-0.3	0.89
⋮	⋮	⋮
H_{16}	0.55	0.01

F1 AI Racing: Evolution & Fitness

Der Lernprozess (Genetischer Algorithmus)

- ▶ **Evaluation:** Die Autos fahren. Wer das Gras berührt, scheidet aus. Am Ende bekommt jedes Auto eine Fitness-Note.
- ▶ **Selektion (80/20-Regel):** Die schlechtesten 80% sterben aus. Die besten 20% (Eliten) überleben den Generationenwechsel.
- ▶ **Mutation:** Die Eliten werden geklont, aber ihre "Gehirn-Gewichte" werden zufällig minimal abgewandelt. So probieren die Kinder automatisch neue Manöver!

Fitness-Design (Belohnung vs. Bestrafung):

$$\text{Fitness} = (\text{Checkpoints} \cdot 600) + (\text{Überholmanöver} \cdot 2500) + \text{PosBonus} - (\text{Zeit} \cdot 0.6) - (\text{Crashes} \cdot 2000) + (\text{Leben} \cdot 0.02)$$

- ▶ **Strategie:** Taktisches Überholen und Fortschritt wird extrem stark gewichtet. Sinnloses Crashten wird hart bestraft, sodass dumme Fahrstile aussterben.

F1 AI Racing: Realistische Fahrphysik

Damit das Training realistisch ist, steuern die Netze ein physikalisches Modell, nicht nur einfache Pixel.

- **Luftwiderstand & Reibung:**

Die Geschwindigkeit reduziert sich in jedem Frame durch Reibung, das Gaspedal muss aktiv genutzt werden.

$$Speed_{neu} = Speed_{alt} \cdot 0.974$$

- **Grip (Untersteuern):**

Wer mit Höchstgeschwindigkeit fährt, verliert massiv an Lenkfähigkeit. Das Netz *muss* lernen, vor scharfen Kurven aktiv den Fuß vom Gas zu nehmen.

$$Grip = 1.0 - \left(\frac{Speed}{MaxSpeed} \right) \cdot 0.32$$

Weitere Projekte im Portfolio

1. Tetris (Reinforcement Learning)

- ▶ Ein in C programmierter RL-Agent, der mittels **Tabular Q-Learning** lernt, Reihen abzubauen und das Spielfeld strategisch flach zu halten.

2. Flappy Bird (Neuroevolution)

- ▶ Eine Java-Simulation, bei der ein **Neuronales Netz** per Neuroevolution lernt, mithilfe von Distanz-Sensoren fehlerfrei durch generierte Röhren zu fliegen.

3. Black Jack (Neuronales Netz / RL)

- ▶ Ein in C entwickeltes Modell, das basierend auf der eigenen Hand und der Karte des Dealers selbstständig die optimale Blackjack-Strategie erlernt.

4. Katzen vs. Hunde Klassifikation (CNN)

- ▶ Ein **Convolutional Neural Network** zur Bilderkennung. Architektur (Faltung, Pooling, Backpropagation) in Python entwickelt.

Unser GitHub-Repository mit allen Projekten und dem vollständigen Quellcode finden Sie unter:

`https://github.com/RechteHand/CI`